

C3M3_peer_review

August 18, 2024

1 C3M3: Peer Reviewed Assignment

1.0.1 Outline:

The objectives for this assignment:

1. Implement kernel smoothing in R and interpret the results.
2. Implement smoothing splines as an alternative to kernel estimation.
3. Implement and interpret the loess smoother in R.
4. Compare and contrast nonparametric smoothing methods.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

```
[2]: # Load Required Packages
library(ggplot2)
library(mgcv)
```

Loading required package: nlme

This is mgcv 1.8-31. For overview type 'help("mgcv-package")'.

2 Problem 1: Advertising data

The following dataset contains measurements related to the impact of three advertising medias on sales of a product, P . The variables are:

- **youtube**: the advertising budget allocated to YouTube. Measured in thousands of dollars;
- **facebook**: the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- **newspaper**: the advertising budget allocated to a local newspaper. Measured in thousands of dollars.

- **sales:** the value in the i^{th} row of the sales column is a measurement of the sales (in thousands of units) for product P for company i .

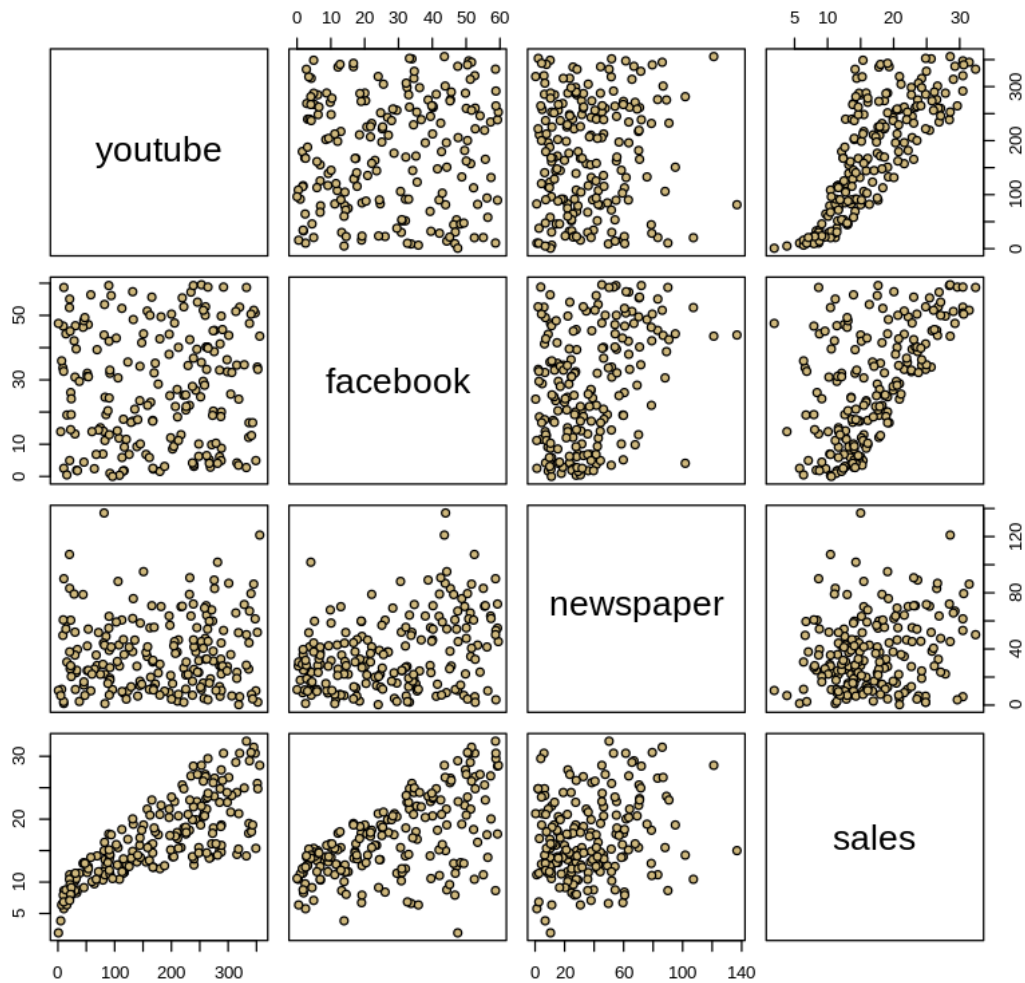
The advertising data treat “a company selling product P ” as the statistical unit, and “all companies selling product P ” as the population. We assume that the $n = 200$ companies in the dataset were chosen at random from the population (a strong assumption!).

First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

```
[3]: # Load in the data
marketing = read.csv("marketing.txt", sep="")
summary(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

youtube	facebook	newspaper	sales
Min. : 0.84	Min. : 0.00	Min. : 0.36	Min. : 1.92
1st Qu.: 89.25	1st Qu.: 11.97	1st Qu.: 15.30	1st Qu.: 12.45
Median : 179.70	Median : 27.48	Median : 30.90	Median : 15.48
Mean : 176.45	Mean : 27.92	Mean : 36.66	Mean : 16.83
3rd Qu.: 262.59	3rd Qu.: 43.83	3rd Qu.: 54.12	3rd Qu.: 20.88
Max. : 355.68	Max. : 59.52	Max. : 136.80	Max. : 32.40

Marketing Data



```
[4]: set.seed(1771) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the
  ↳ data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indices to
  ↳ be included in the training set

train_marketing = marketing[index, ] #set the training set to be the randomly
  ↳ sampled rows of the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining
  ↳ rows
dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

1. 40 2. 4

1. 160 2. 4

1.(a) Working with nonlinearity: Kernel regression

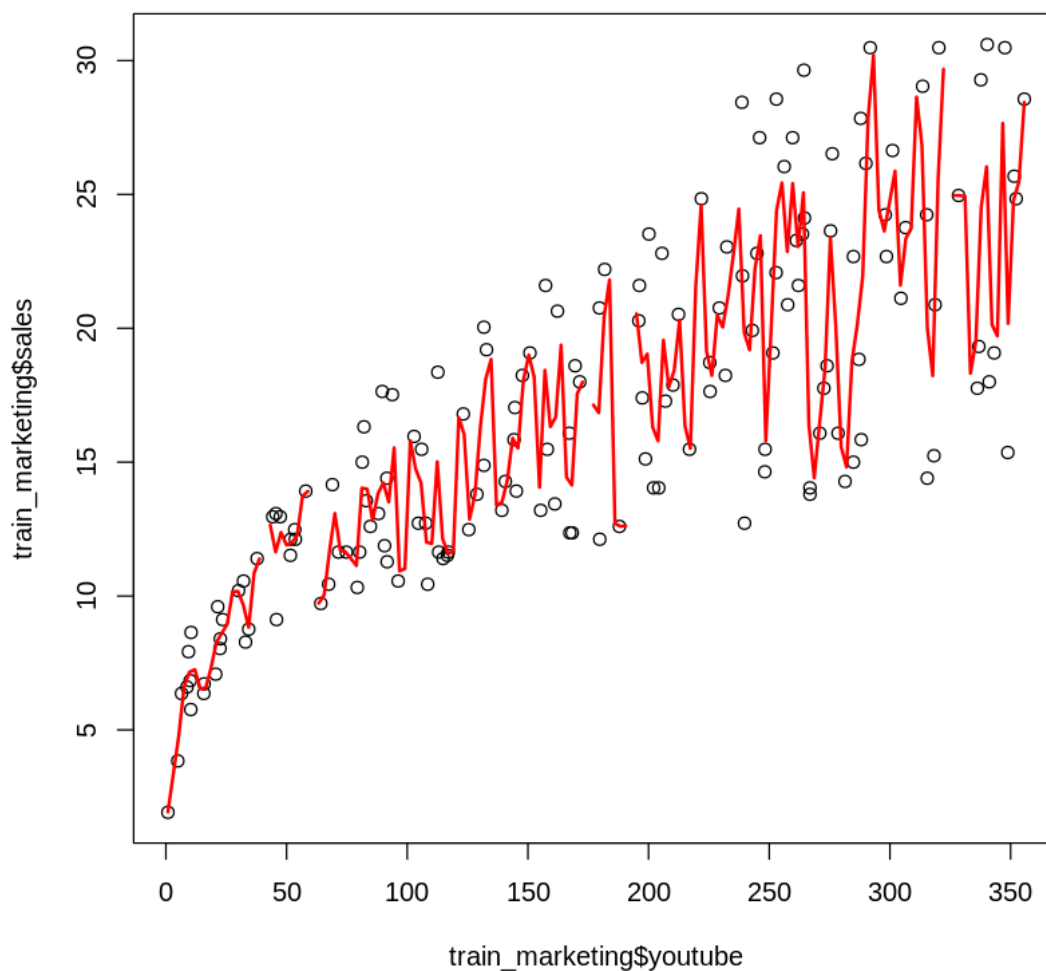
Note that the relationship between `sales` and `youtube` is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. For now, let's just focus on the relationship between `sales` and `youtube`, omitting the other variables (future lessons on generalized additive models will allow us to bring back other predictors).

Using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor), and then fit and overlay a kernel regression. Experiment with the bandwidth parameter until the smooth looks appropriate, or comment why no bandwidth is ideal. Justify your answer.

```
[5]: # Perform kernel smoothing with varying bandwidths
ssk2 = ksmooth(train_marketing$youtube, train_marketing$sales, 'normal',
  ↪bandwidth = 2)
ssk3 = ksmooth(train_marketing$youtube, train_marketing$sales, 'normal',
  ↪bandwidth = 3)
ssk4 = ksmooth(train_marketing$youtube, train_marketing$sales, 'normal',
  ↪bandwidth = 4)
overlay_y = (ssk2$y + ssk3$y + ssk4$y) / 3

plot(train_marketing$youtube, train_marketing$sales)

# Overlay the combined kernel smoothed line
lines(ssk2$x, overlay_y, col="red", lwd=2)
```



3 1-a My answer

The plot shows that the chosen bandwidth was too small, made the smooth spline to overfit the data by capturing excessive noise and fluctuations. There is no ideal bandwidth would smooth out these variations, revealing the underlying trend without reacting to every minor deviation. I failed to find a bandwidth that would help balance the trade-off between bias and variance, providing a clearer and more generalizable representation of the relationship between YouTube spending and sales. Mission impossible.

1.(b) Working with nonlinearity: Smoothing spline regression

Again, using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor). This

time, fit and overlay a smoothing spline regression model. Experiment with the smoothing parameter until the smooth looks appropriate. Explain why it's appropriate and justify your answer.

```
[7]: # Fit a smoothing spline regression model
spline_model <- smooth.spline(train_marketing$youtube, train_marketing$sales)
plot(train_marketing$youtube, train_marketing$sales, main = "Smoothing Spline
  ↳Regression", pch = 21, bg = c("#CFB87C"))
lines(spline_model, col="purple", lwd=2)

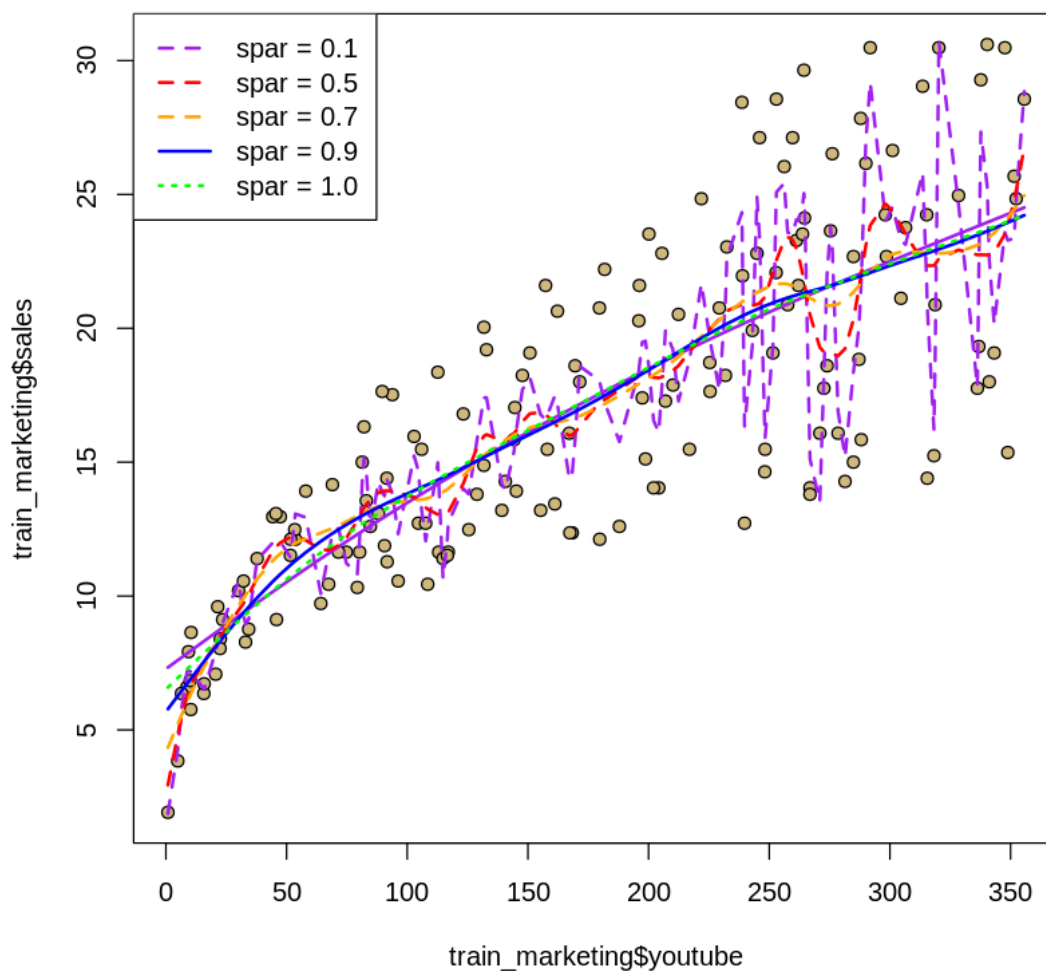
# Plot the original data
#plot(train_marketing$youtube, train_marketing$sales, main = "Experimenting
  ↳with Smoothing Parameters", pch = 21, bg = c("#CFB87C"))

# Fit and overlay smoothing spline with different parameters
spline_01 <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar
  ↳= 0.1)
spline_05 <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar
  ↳= 0.5)
spline_07 <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar
  ↳= 0.7)
spline_09 <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar
  ↳= 0.9)
spline_1 <- smooth.spline(train_marketing$youtube, train_marketing$sales, spar
  ↳= 1)

# Overlay the smoothing splines with different colors
lines(spline_01, col="purple", lwd=2, lty=2) # spar = 0.1
lines(spline_05, col="red", lwd=2, lty=2)    # spar = 0.5
lines(spline_07, col="orange", lwd=2, lty=2)  # spar = 0.7
lines(spline_09, col="blue", lwd=2, lty=1)    # spar = 0.9
lines(spline_1, col="green", lwd=2, lty=3)    # spar = 1.0

# Add a legend to the plot
legend("topleft",
  legend = c("spar = 0.1", "spar = 0.5", "spar = 0.7", "spar = 0.9", "spar
    ↳= 1.0"),
  col = c("purple", "red", "orange", "blue", "green"),
  lty = c(2, 2, 2, 1, 3), lwd = 2)
```

Smoothing Spline Regression



4 1-b My answer

The plot shows that a `spar` value of 0.1 is not appropriate, as it results in a very wiggly line that overfits the noise in the data. The `spar` values of 0.9 and 1.0 provide smoother, more generalized fits, but the line with `spar` = 1.0 might be slightly oversmoothed, missing some of the data's nuances. A `spar` value of 0.7 seems most appropriate, striking a good balance between capturing the underlying trend and avoiding overfitting.

1.(c) Working with nonlinearity: Loess

Again, using the `train_marketing` set, plot `sales` (response) against `youtube` (predictor). This time, fit and overlay a loess regression model. You can use the `loess()` function in a similar

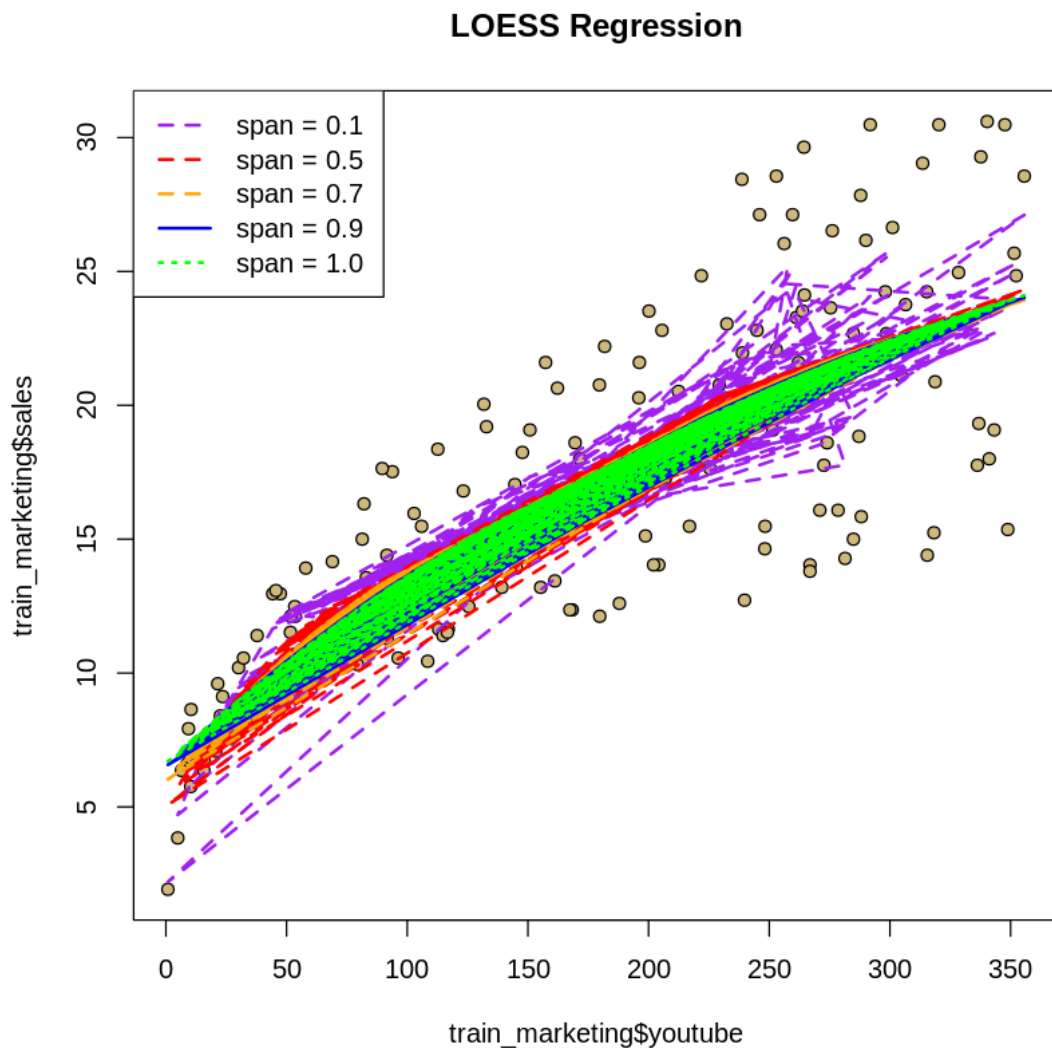
way as the `lm()` function. Experiment with the smoothing parameter (`span` in the `geom_smooth()` function) until the smooth looks appropriate. Explain why it's appropriate and justify your answer.

```
[8]: # Fit LOESS models with different spans
loess_01 <- loess(sales ~ youtube, data = train_marketing, span = 0.1)
loess_05 <- loess(sales ~ youtube, data = train_marketing, span = 0.5)
loess_07 <- loess(sales ~ youtube, data = train_marketing, span = 0.7)
loess_09 <- loess(sales ~ youtube, data = train_marketing, span = 0.9)
loess_1 <- loess(sales ~ youtube, data = train_marketing, span = 1.0)

plot(train_marketing$youtube, train_marketing$sales, main = "LOESS Regression",
      pch = 21, bg = c("#CFB87C"))

lines(train_marketing$youtube, predict(loess_01), col = "purple", lwd = 2, lty = 2)
lines(train_marketing$youtube, predict(loess_05), col = "red", lwd = 2, lty = 2)
lines(train_marketing$youtube, predict(loess_07), col = "orange", lwd = 2, lty = 2)
lines(train_marketing$youtube, predict(loess_09), col = "blue", lwd = 2, lty = 1)
lines(train_marketing$youtube, predict(loess_1), col = "green", lwd = 2, lty = 3)

legend("topleft",
      legend = c("span = 0.1", "span = 0.5", "span = 0.7", "span = 0.9", "span = 1.0"),
      col = c("purple", "red", "orange", "blue", "green"),
      lty = c(2, 2, 2, 1, 3), lwd = 2)
```

5 1-c My answer

The plot shows that a **span** of 0.1 is the worst, as it overfits the data, capturing too much noise and resulting in a wiggly line. On the other hand, a **span** of 0.7 appears to be the optimal choice, as it balances smoothness and accuracy, capturing the overall trend without being overly sensitive to minor fluctuations. The **span** of 1.0 is too smooth, missing some of the variability in the data and potentially underfitting.

1.(d) A prediction metric

Compare the models using the mean squared prediction error (MSPE) on the **test_marketing** dataset. That is, calculate the MSPE for your kernel regression, smoothing spline regression, and

loess model, and identify which model is best in terms of this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k} \sum_{i=1}^k (y_i^* - \hat{y}_i^*)^2$$

where y_i^* are the observed response values in the test set and $\hat{y}_i^*$ are the predicted values for the test set (using the model fit on the training set).

*Note that `ksmooth()` orders your designated `x.points`. Make sure to account for this in your MSPE calculation.

```
[9]: kernel_pred <- ksmooth(test_marketing$youtube, test_marketing$sales, bandwidth_
  ↪ = 30, kernel = "normal", x.points = test_marketing$youtube)
mspe_kernel <- mean((test_marketing$sales - kernel_pred$y)^2)

# Spline Model - Prediction on test data
spline_pred <- predict(spline_model, test_marketing$youtube)
mspe_spline <- mean((test_marketing$sales - spline_pred$y)^2)

# LOESS Model - Prediction on test data (assuming span = 0.7)
loess_pred <- predict(loess_07, test_marketing$youtube)
mspe_loess <- mean((test_marketing$sales - loess_pred)^2)

mspe_kernel
mspe_spline
mspe_loess
```

58.1119182671533

17.5389997021376

18.0768119832863

6 1-d My answer

The Spline Model has the lowest MSPE (17.54), indicating it is the best model among the three for predicting sales on the `test_marketing` data. It generalizes better to new data, resulting in more accurate predictions

7 Problem 2: Simulations!

Simulate data (one predictor and one response) with your own nonlinear relationship. Provide an explanation of how you generated the data. Then answer the questions above (1.(a) - 1.(d)) using

your simulated data.

8 Problem 2

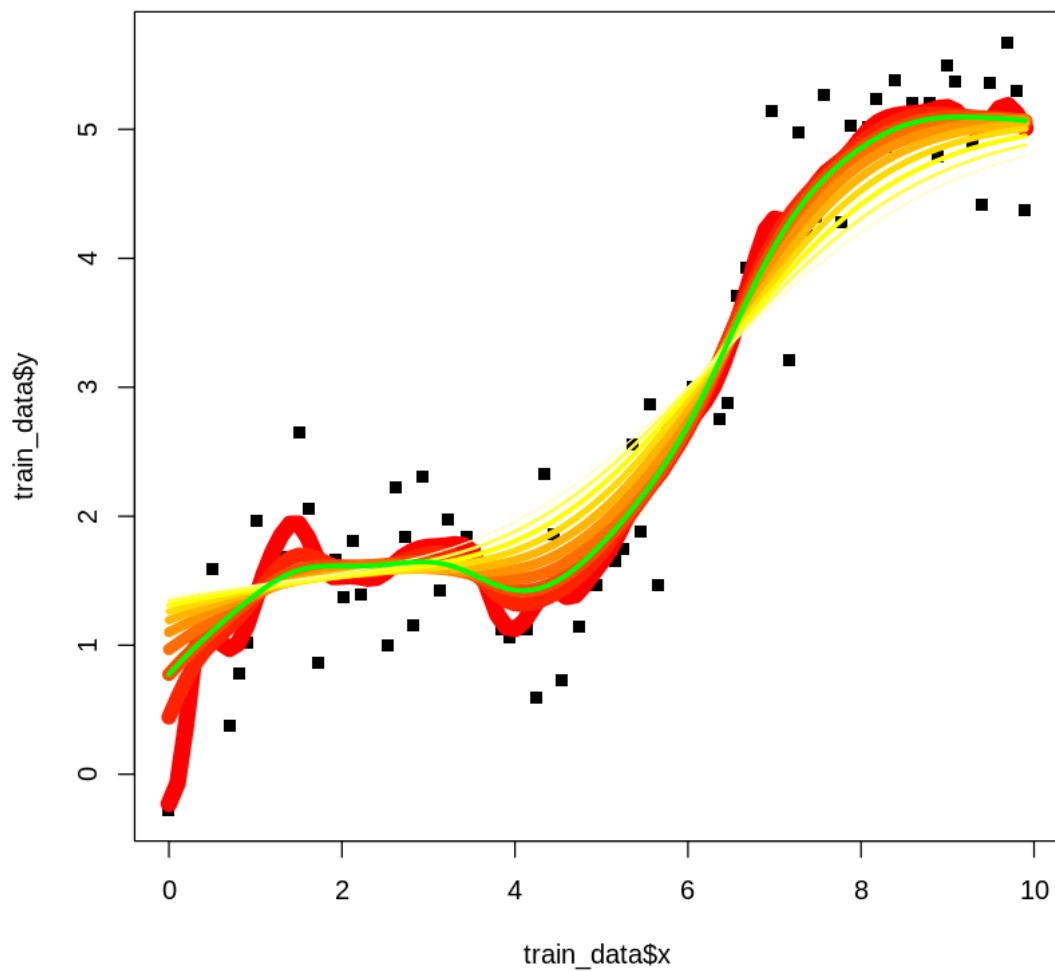
I chose the equation ($y = \sin(x) + 0.5x + \text{random noise}$) because it combines a nonlinear sine component with a linear trend, making it ideal for testing the flexibility of different regression models. The added random noise simulates real-world data variability, challenging the models to capture the underlying pattern without overfitting.

```
[11]: #simulated data
set.seed(123)
n <- 100
x <- seq(0, 10, length.out = n)

y <- sin(x) + 0.5 * x + rnorm(n, mean = 0, sd = 0.5)
simulated_data <- data.frame(x = x, y = y)
train_index <- sample(seq_len(n), size = 0.8 * n)
train_data <- simulated_data[train_index, ]
test_data <- simulated_data[-train_index, ]

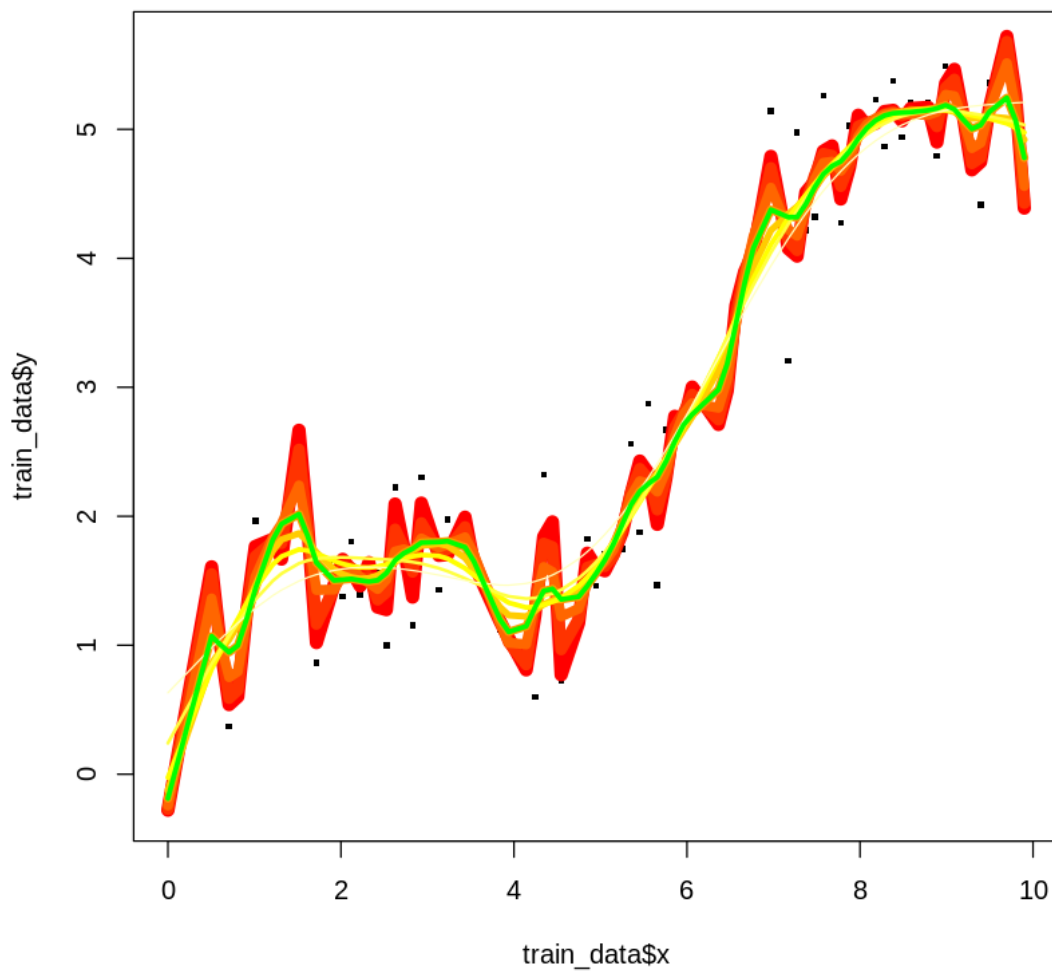
[21]: #1.a
bws <- seq(0.5, 5, length.out = 10)
cols <- heat.colors(length(bws))
plot(train_data$x, train_data$y, pch = 15, cex = 1, main = "Kernel Smoothing")
for (i in 1:length(bws)) {
  mod <- ksmooth(train_data$x, train_data$y, 'normal', bandwidth = bws[i])
  lines(mod$x, mod$y, col = cols[i], lwd = 11 - i)
}
best_bw <- 1.5
modfin <- ksmooth(train_data$x, train_data$y, 'normal', bandwidth = best_bw)
lines(modfin$x, modfin$y, col = "green", lwd = 3)
```

Kernel Smoothing

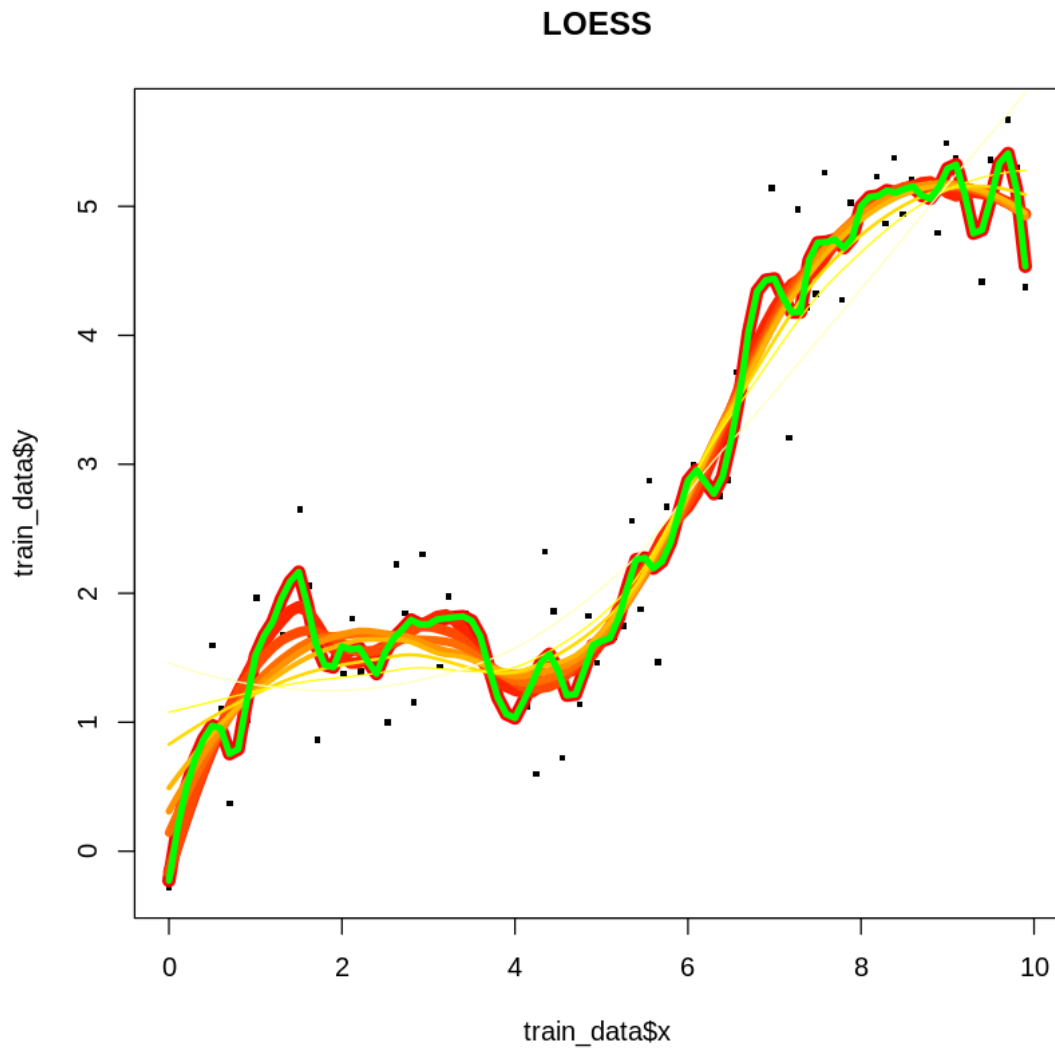


```
[22]: #1.b
spans <- seq(0.1, 0.8, 0.1)
cols <- heat.colors(length(spans))
plot(train_data$x, train_data$y, pch = 15, cex = 0.5, main = "Smoothing_
↳ Splines")
for (i in 1:length(spans)) {
  mod <- smooth.spline(train_data$x, train_data$y, spar = spans[i])
  lines(mod$x, mod$y, col = cols[i], lwd = 9 - i)
}
best_spar <- 0.4
modssfin <- smooth.spline(train_data$x, train_data$y, spar = best_spar)
lines(modssfin$x, modssfin$y, col = "green", lwd = 3)
```

Smoothing Splines



```
[23]: #1.c
ll.x <- seq(min(train_data$x), max(train_data$x), length.out = 100)
spans <- seq(0.1, 1, 0.1)
cols <- heat.colors(length(spans))
plot(train_data$x, train_data$y, pch = 15, cex = 0.5, main = "LOESS")
for (i in 1:length(spans)) {
  lomod <- loess(y ~ x, data = train_data, span = spans[i])
  lines(ll.x, predict(lomod, ll.x), col = cols[i], lwd = 9 - i)
}
best_span <- 0.1
lomod <- loess(y ~ x, data = train_data, span = best_span)
lines(ll.x, predict(lomod, ll.x), col = "green", lwd = 4)
```



```
[24]: #1.d
# Refit LOESS with a reasonable span
best_span <- 0.2
lomod <- loess(y ~ x, data = train_data, span = best_span)

# Make predictions on the test set
loess_pred <- predict(lomod, newdata = data.frame(x = test_data$x))

# Calculate MSPE for the LOESS model
loessMSPE <- mean((test_data$y - loess_pred)^2, na.rm = TRUE)
cat("\n Loess MSPE:", loessMSPE)
```

```

kernel_pred <- ksmooth(test_data$x, test_data$y, bandwidth = best_bw, kernel =
  ↪ "normal", x.points = test_data$x)
kernelMSPE <- mean((test_data$y - kernel_pred$y)^2)
cat("\n Kernel MSPE:", kernelMSPE)
spline_pred <- predict(modssfin, test_data$x)
splineMSPE <- mean((test_data$y - spline_pred$y)^2)
cat("\n Smooth Spline MSPE:", splineMSPE)
loess_pred <- predict(lomod, test_data$x)
loessMSPE <- mean((test_data$y - loess_pred)^2)

```

Loess MSPE: 0.1284245

Kernel MSPE: 0.1128654

Smooth Spline MSPE: 0.1499076

1a: Kernel smoothing was tested with multiple bandwidths, and the best bandwidth is selected and plotted.

1b: Smoothing splines with different spar values are evaluated, and the most appropriate one is selected.

1c: LOESS regression was performed with different span values, and the optimal span is selected.

1d: The MSPE is calculated for each model on the test data, and the model with the lowest MSPE is identified as the best fitting model.

[]: